

# PMorph: A Task-Specialized Robot Manipulator Morphology Optimizer

Stephen Gillet<sup>1</sup>

**Abstract**—This paper seeks to address the overactuation, energy-inefficiency, and lack of task specialization seen in manufacturing from the current suite of generalized robotic manipulators. The approach involves specifying a task in a MuJoCo simulated environment and then using a PPO agent to optimize the morphology for that environment given a reward function. The morphological elements to be optimized include the number of links, the lengths of each link, and the type of joints involved. The reward function is tuned to weigh energy cost, end effector movement, end effector accuracy, manipulability measures, number of links, and task completion. PPO is ideal for this task because it can efficiently optimize over the continuous action space that is morphology, resulting in a robot that is more simple, efficient, and capable than those commonly used in industry today.

## I. INTRODUCTION

Imagine a factory where every employee has to be trained on and work every job in the plant. Imagine how inefficient that would be. Every employee would have general knowledge and ability, but no one would have a chance to specialize and become very good at any particular job. This is exactly the state of robotics in manufacturing today where high costs and lack of tailorability are some of the most cited problems for the adoption of robotics [1][2]. To make matters worse [3] and [4] show that most manipulators are overactuated, inefficient, and ineffective for most tasks they are used in today.

This paper seeks to address these issues by using a PPO agent to optimize the morphology of a robotic manipulator to the particular task it is being used for. This ensures that the robot is as simple, efficient, and capable as possible for its particular job while still allowing the designer to specify the level of robustness.

PPO was chosen for its ability to explore the continuous action space that is morphology and it has been shown to do so efficiently and robustly [5]. The core method is to use Stable Baselines3 PPO implementation to iterate on the design parameters (number of links, joint types (X, Y, Z hinge, and Z actuation), and link lengths), use MuJoCo to simulate that iteration on a simple task using RRT-Connect and damped least squares (DLS) inverse kinematics (IK) for the path planning, feed the results (success, efficiency, and manipulability index) back into the PPO model. RRT-Connect was chosen for simplicity and it has been shown to be faster than other methods [6] which enables faster and more robust training. DLS IK was chosen for a similar

reason, it has been shown to be robust especially in situations where the target position is out of reach [7] which is inevitable when exploring morphologies, some of which will be too small or otherwise poorly suited to the task as the agent learns.

The evaluation signals will be successful path planning, path length, energy cost, and manipulability index. The manipulability index is a measure of a manipulators ability to arbitrarily change the position and orientation of the end effector at a particular point, borrowed from [8]. The tasks will be kept simple but varied in ways to test different types of movements like moving near the base, moving from near to far, different elevations, more linear movements, and different obstacle constraints. One might imagine that a lot of pick and place tasks are mostly linear and so might largely be done by linear actuators more efficiently for example.

The research questions to be answered: Can PPO be used to generate more efficient morphologies? Can morphologies be generated for tasks that generalized manipulators cannot do or cannot perform well? Can more simple morphologies (less DOF) be generated that can perform the same tasks with similar levels of accuracy?

## II. RELATED WORK

Most of the research in this area codesigns morphology and control of atomic, modular robots. [9] shows that PPO can be used to generate freeform morphologies optimized for locomotion tasks from a set of atomic modules. [10] shows that reinforcement learning can be used on a modular robot to co-optimize morphology and control and outperform optimized control on a fixed morphology. [11] also uses reinforcement learning to co-optimize with the addition of a pretrained foundation model to accelerate the training process and improve results.

Most pertinent to this paper, [12] shows that PPO and A3C can be used to generate optimal morphologies that improve performance of reconfigurable robots, [13] uses DDQN to co-optimize morphology and control for modular robotic manipulators, and [14] uses soft actor critic for co-optimization showing that the approach resulted in drastically less design prototypes. This background motivates the approach of this paper which is going to use simple controllers and similar reinforcement learning techniques to maximize morphological potential. FANUC 6-DOF series of manipulators are the most commonly used in industry today [15][16] and so their morphology will be used as a baseline for comparison. The other baseline will be the 7-DOF Franka Emika Panda because it is considered a standard in research

\*This work was not supported by any organization

<sup>1</sup>Stephen Gillet is a Master's student with the Robotic Program, University of Colorado Boulder, Boulder, CO 80309, U.S.A. stgi5837@colorado.edu

[17][18]. Given the same environment, task, and controllers which morphologies will perform better?

### III. PROBLEM FORMULATION

The task is formulated as a single-step reinforcement learning problem in a MuJoCo simulated environment. A morphology  $\theta$  is chosen, run through the simulation, and then evaluated based on weighted reward function terms.

#### A. Morphology Parameterization

A candidate morphology is defined by the tuple

$$\theta = (n, l, j),$$

where

- $n \in \{2, \dots, 7\}$  is the number of links,
- $l = [l_1, \dots, l_n] \in [0.05, 1.2]^n$  are the link lengths (m),
- $j = [j_1, \dots, j_n] \in \{0, 1, 2, 3\}^n$  are the joint types (0: revolute about x, 1: about y, 2: about z, 3: slide in z).

The action space of the RL agent is the continuous box

$$\mathcal{A} = [0, 1]^{1+7+7},$$

from which  $n$ ,  $l$ , and  $j$  are decoded.

#### B. Task Specification

The robot has to move from start poses to goal poses (position + unit quaternion) in a cluttered workspace with fixed obstacles (walls, shelves, ceilings). An example of such an environment may be seen in Figure 1, which shows the container task where two sets of start and goal poses are set at either ends of a shipping-container-like workspace with the arm positioned in the middle. The poses are modified with Gaussian noise ( $\sigma = 0.05$ ) at evaluation in order to encourage robust designs.

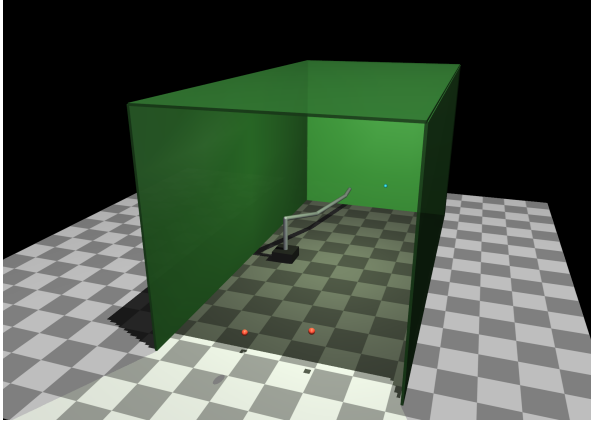


Fig. 1. Container task. Start and goal positions represented with blue and red dots surrounded by wall and ceiling obstacles.

#### C. Evaluation Pipeline

Morphology  $\theta$  is evaluated as follows:

- 1) A MuJoCo XML model is generated with  $\theta$  and the task poses/obstacles.
- 2) Inverse Kinematics is solved for the start and goal poses using Damped Least Squares.

- 3) A path between the start and goal configurations is computed using RRT-Connect.
- 4) If a path connect be found, return reward  $30 - 100 \times \text{IK error}$ .
- 5) Otherwise, calculate the other reward terms and return the weighted sum:
  - End-effector path length  $p = \sum \|\mathbf{x}_{ee}^{t+1} - \mathbf{x}_{ee}^t\|$  (Euclidean distance between each point along path),
  - Manipulability  $\mu = \min(\sigma(\mathbf{J}))$  (minimum singular value of manipulability jacobian at start and goal [19]),
  - Energy  $E = \sum \|\boldsymbol{\tau} \odot \mathbf{v}\| \cdot \Delta t$  (MuJoCo inverse dynamics using constant velocity segments along path at torque limits).

#### D. Reward Function

The scalar reward is

$$r(\theta) = \frac{1}{N} \sum_{i=1}^N \left[ w_0 \cdot \text{success}_i - w_1 \cdot p_i - w_2 \cdot (e_{\text{start},i} + e_{\text{goal},i}) + w_3 \cdot (\mu_{\text{start},i} + \mu_{\text{goal},i}) - w_4 \cdot (n - 2) - w_5 \cdot E_i \right] \quad (1)$$

where  $N$  is the number of start-goal pairs,  $e$  is IK error, success is valid path, and the weights ( $w_k$ ) are tuned to balance the competing objectives (accuracy vs. efficiency vs. manipulability). The final optimization problem is therefore

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\pi(\cdot|\phi)} [r(\theta)],$$

where  $\pi$  is the PPO policy given the parameters  $\phi$ .

This formulation encodes the challenges outlined in Section 1:

- 1) Energy efficiency via the energy term.
- 2) Task specialization via the success, accuracy, and manipulability terms.
- 3) And simplicity via the link number and path length terms.

### IV. METHOD

#### A. PPO Action Decoding

The PPO agent selects an action  $a \in [0, 1]^{15}$  which is then decoded into morphology parameters  $\theta$  as shown in Algorithm 1.

---

#### Algorithm 1 Morphology Decoding from PPO Action

---

- 1: **Input:** action  $a \in [0, 1]^{15}$
  - 2:  $n \leftarrow \text{round}(a_0 \cdot (7 - 2) + 2)$  ▷ links: 2–7
  - 3:  $l \leftarrow a_{1:8} \cdot (1.2 - 0.05) + 0.05$  ▷ lengths (m)
  - 4:  $j \leftarrow \text{round}(a_{8:15} \cdot 3)$  ▷ types: 0–3 (hinge X/Y/Z, slide Z)
  - 5: **Return:**  $\theta = (n, l_{:n}, j_{:n})$
-

## B. Simulation

$\theta$  is then passed into the evaluation environment, which is implemented as a Gymnasium class ‘robotArmEnv’.

- 1) **XML Generation:** A dynamic MuJoCo model is built via ‘generateXML( $n, l, j$ )’, which assembles the links sequentially according to the parameter specifications using revolute/slide joints and capsule geoms (mass =  $l_i$ ). Fixed obstacles (walls, ceiling, shelves, floor) are included as per the task set up.
- 2) **Path Planning:** Damped least-squares IK (robust variant with collision-aware line search) solves for start/goal configurations. RRT-Connect then finds a collision-free joint-space path [6], using constant-velocity segments for energy estimation via MuJoCo inverse dynamics.
- 3) **Reward Computation:**  $r(\theta)$  from Eq. (1) is returned with  $N$  being the number of sets of start and goal poses in the task and noise ( $\sigma = 0.05$ ) added to poses for robustness.

## C. Training

Training follows the standard PPO loop over 1M timesteps (single-step episodes):

---

### Algorithm 2 PMorph Training

---

- 1: Initialize PPO policy  $\pi_\phi$
- 2: **for** each episode  $t = 1$  to 1M **do**
- 3:   Sample  $\theta \sim \pi_\phi(\cdot|\phi)$
- 4:    $r \leftarrow \text{eval}(\theta)$  ▷ Section III pipeline
- 5:   Store  $(\theta, r)$  in buffer
- 6:   Update  $\phi$  via the clipped surrogate loss [5]:

$$\begin{aligned} \mathcal{L}(\phi) = & \hat{\mathbb{E}}_t \left[ \min(r_t(\phi) \hat{A}_t, \right. \\ & \left. \text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \\ & + S[\pi_\phi](s_t) - \beta \cdot \mathbb{E}_t[(\hat{V}_\phi(s_t) - V_t^{\text{targ}})^2] \\ & \quad \triangleright \epsilon = 0.2, \text{ GAE advantages [20]} \end{aligned}$$

- 7: **end for**
- 

Hyperparameters:  $\epsilon = 0.2$  (clip ratio) and  $\beta = 0.5$  (value loss weight)

## D. Evaluation

After training, the optimized morphology  $\theta^*$  is evaluated on a test set of start/goal pairs with noise. The reward terms are averaged over that test set to assess performance. A comparison is made using the baseline Panda and FANUC-like arm morphology. Based on the most commonly used model (FANUC LR Mate 200iD [21]), the baseline has 6 links, with lengths of 0.165, 0.330, 0.08, 0.285, 0.05, 0.05 (meters), and joint types of 2, 0, 0, 2, 0, 2 (revolute about z, x, x, z, x, z). The Panda approximation similarly taken from the product manual [22] has 7 links, lengths of 0.333, 0.316, 0.0825, 0.0825, 0.384, 0.107, 0.05 (meters), and joint types of 2, 0, 2, 0, 2, 0, 2 (revolute about z, x, z, x, z, x, z).

## V. EXPERIMENTAL SETUP

To evaluate PMorph, the MuJoCo (version 3.3.7) physics engine was used across a set of tasks specified below using the reward function defined in Equation (1) and the evaluation pipeline described in Section III.

### A. Tasks

5 tasks were chosen to test different types of motions (different axes, rotations, and positions w.r.t. the base) and obstacle constraints to create a thorough and practical evaluation. All tasks use a workspace centered at the robot base (0, 0, 0) except for the ‘‘Wall Mount’’ task where the base is moved to the middle of the near wall (0, -0.4, 1.0) and rotated  $\pi/2$  about the x-axis to face outward from the wall. The tasks are as follows:

- 1) **Container Task:** The robot is positioned on the floor of a shipping-container-like environment in the middle and slightly to one side with walls and a ceiling (2m x 1m x 1m) all around and one open end. The start poses are near and facing the back of the container and the goal poses toward and near the open end (see Figure 1):

TABLE I  
CONTAINER TASK POSES

| Name    | Position         | Quaternion              |
|---------|------------------|-------------------------|
| Start 1 | [-1.8, 0.3, 0.3] | [0.7071, 0, -0.7071, 0] |
| Goal 1  | [1.9, 0.9, 0.4]  | [0.7071, 0, 0.7071, 0]  |
| Start 2 | [-1.8, 0.8, 0.4] | [0.7071, 0, -0.7071, 0] |
| Goal 2  | [1.8, 0.31, 0.2] | [0.7071, 0, 0.7071, 0]  |

TABLE II  
CONTAINER TASK OBSTACLES

| Name       | Position         | Size             |
|------------|------------------|------------------|
| Back Wall  | [-2.0, 0.6, 1.0] | [0.01, 1.0, 1.0] |
| Left Wall  | [0, -0.4, 1.0]   | [2.0, 0.01, 1.0] |
| Right Wall | [0, 1.6, 1.0]    | [2.0, 0.01, 1.0] |
| Ceiling    | [0, 0.6, 2.0]    | [2.0, 1.0, 0.01] |

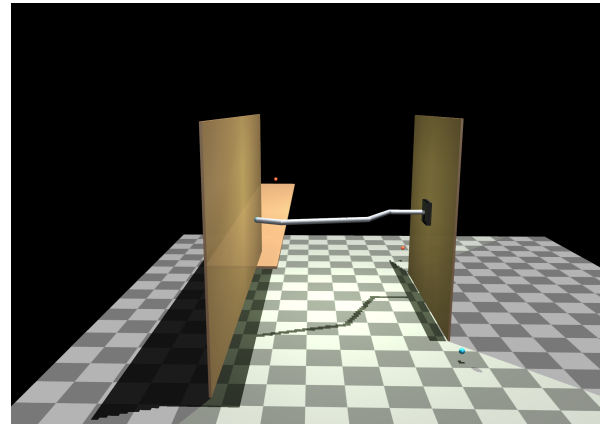


Fig. 2. Wall Mount task.

- 2) **Wall Mount Task:** The robot is mounted on the middle of a wall facing an opposing wall with a shelf mounted

in the middle. One start pose is to the side of the mount wall facing down and the corresponding goal pose is on top of the shelf facing toward that wall. The other set is the opposite where the start pose is on the shelf facing the wall and the goal is to the side facing down (Figure 2):

TABLE III  
WALL MOUNT TASK POSES

| Name    | Position           | Quaternion               |
|---------|--------------------|--------------------------|
| Start 1 | [-1.4, -0.4, 0.1]  | [-0.7071, -0.7071, 0, 0] |
| Goal 1  | [1.75, 1.36, 1.11] | [1, 0, 0, 0]             |
| Start 2 | [-0.9, 1.35, 1.1]  | [1, 0, 0, 0]             |
| Goal 2  | [1.5, -0.4, 0.2]   | [-0.7071, -0.7071, 0, 0] |

TABLE IV  
WALL MOUNT TASK OBSTACLES

| Name       | Position         | Size              |
|------------|------------------|-------------------|
| Mount Wall | [0, -0.4, 1.0]   | [1.0, 0.01, 1.0]  |
| Shelf Wall | [0, 1.6, 1.0]    | [2.0, 0.01, 1.0]  |
| Shelf      | [0.0, 1.35, 1.0] | [2.0, 0.25, 0.01] |

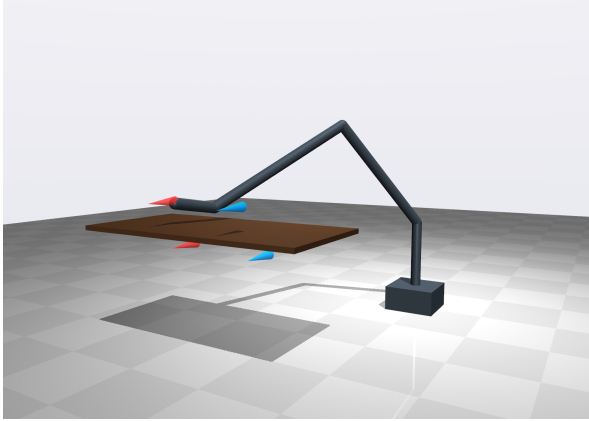


Fig. 3. Shelf task.

- 3) **Shelf Task:** A single shelf obstacle is placed off to the side of the robot at 0.5m height. There are two sets of start and goal poses alternately above and below the shelf all facing in the positive x direction.

TABLE V  
SHELF TASK POSES

| Name    | Position           | Quaternion             |
|---------|--------------------|------------------------|
| Start 1 | [1.1, -0.31, 0.6]  | [0.7071, 0, 0.7071, 0] |
| Goal 1  | [0.9, -0.71, 0.4]  | [0.7071, 0, 0.7071, 0] |
| Start 2 | [0.95, -0.29, 0.4] | [0.7071, 0, 0.7071, 0] |
| Goal 2  | [1.05, -0.69, 0.6] | [0.7071, 0, 0.7071, 0] |

TABLE VI  
SHELF TASK OBSTACLES

| Name  | Position         | Size              |
|-------|------------------|-------------------|
| Shelf | [1.0, -0.5, 0.5] | [0.25, 0.5, 0.01] |

- 4) **Near-Base Outreach Task:** No obstacles; motion from near the base outward in one direction (e.g., positive x):

TABLE VII  
NEAR-BASE OUTREACH TASK POSES

| Name  | Position      | Quaternion   |
|-------|---------------|--------------|
| Start | [0.1, 0, 0.1] | [1, 0, 0, 0] |
| Goal  | [1.0, 0, 0.5] | [1, 0, 0, 0] |

- 5) **Side-to-Side Task:** No obstacles; poses on either side of the base (e.g., left and right in y-direction):

TABLE VIII  
SIDE-TO-SIDE TASK POSES

| Name    | Position       | Quaternion   |
|---------|----------------|--------------|
| Start 1 | [0, -0.5, 0.3] | [1, 0, 0, 0] |
| Goal 1  | [0, 0.5, 0.3]  | [1, 0, 0, 0] |
| Start 2 | [0, 0.5, 0.5]  | [1, 0, 0, 0] |
| Goal 2  | [0, -0.5, 0.5] | [1, 0, 0, 0] |

For broader validation, we report results on all tasks, with the container task as the primary benchmark.

### B. Baselines

We compare against two fixed industrial morphologies:

- **FANUC LR Mate 200iD** (most common 6-DoF model, 70,000+ installed [21]):  $n = 6$ , lengths [0.165, 0.330, 0.08, 0.285, 0.05, 0.05] m (scaled from official datasheet), joint types [2, 0, 0, 2, 0, 2] (all revolute).
- **Franka Panda** (7-DoF research standard):  $n = 7$ , lengths  $2 \times [0.333, 0.316, 0.0825, 0.0825, 0.384, 0.088, 0.01]$  m, joint types [2, 1, 2, 0, 2, 0, 2].

All baselines use identical IK, planning, and reward computation for fair comparison.

### C. Metrics

Performance is measured by the average reward  $r(\theta)$  over 100 test episodes (with fresh noise). We also report component-wise breakdowns:

- Success rate: fraction of valid IK + RRT paths.
- Path length  $p$  (m).
- Energy  $E$  (J).
- Manipulability  $\mu$  (min singular value).
- Link count  $n$ .

Results are averaged over 3 random seeds with 95% confidence intervals.

### D. Implementation and Compute

Training used Stable Baselines3 PPO (1M timesteps, single-step episodes) on an NVIDIA RTX 4060 GPU (or CPU fallback: 11th Gen Intel i9-11900KF). The environment was vectorized with 24 parallel instances ('SubprocVecEnv'). Full hyperparameters are in Table IX.

TABLE IX  
PPO HYPERPARAMETERS

| Parameter                          | Value |
|------------------------------------|-------|
| Learning rate                      | 0.001 |
| Horizon ( $n_{\text{steps}}$ )     | 256   |
| Batch size                         | 512   |
| Epochs per update                  | 4     |
| Discount factor ( $\gamma$ )       | 0.98  |
| Clip ratio ( $\epsilon$ )          | 0.2   |
| Value loss coefficient ( $\beta$ ) | 0.5   |
| Entropy coefficient                | 0.0   |

All code, models, and logs are available in the supplementary material.

## VI. CONCLUSIONS

A conclusion section is not required. Although a conclusion may review the main points of the paper, do not replicate the abstract as the conclusion. A conclusion might elaborate on the importance of the work or suggest applications and extensions.

## APPENDIX

Appendixes should appear before the acknowledgment.

## ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression, "One of us (R. B. G.) thanks . . ." Instead, try "R. B. G. thanks". Put sponsor acknowledgments in the unnumbered footnote on the first page.

References are important to the reader; therefore, each citation must be complete and correct. If at all possible, references should be commonly available publications.

## REFERENCES

- [1] McKinsey and Company, "Industrial robotics: Insights into the sector's future growth dynamics," McKinsey and Company, Tech. Rep., 2019. [Online]. Available: <https://www.mckinsey.com/business-functions/operations/our-insights/industrial-robotics-insights-into-the-sectors-future-growth-dynamics>
- [2] R. V. Patel, "Opportunities and challenges for adoption of robotics in packaging industry: A review," *IOSR Journal of Engineering*, vol. 14, no. 1, pp. 32–38, Jan 2024, available: [https://www.iosrjen.org/Papers/vol14\\_issue1/E1401013238.pdf](https://www.iosrjen.org/Papers/vol14_issue1/E1401013238.pdf).
- [3] M. Russo, L. Raimondi, X. Dong, and D. Axinte, "Task-oriented optimal dimensional synthesis of robotic manipulators with limited mobility," *Robotics and Computer-Integrated Manufacturing*, vol. 69, p. 102098, 2021.
- [4] B. He, S. Wang, and Y. Liu, "Underactuated robotics: A review," *International Journal of Advanced Robotic Systems*, vol. 16, no. 4, pp. 1–14, 2019.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- [6] J. J. Kuffner and S. M. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, 2000, pp. 995–1001 vol.2.
- [7] S. R. Buss, "Introduction to inverse kinematics with jacobian transpose, pseudoinverse and damped least squares methods," *IEEE Journal of Robotics and Automation*, vol. 17, no. 1-19, p. 16, 2004.
- [8] T. Yoshikawa, "Manipulability of robotic mechanisms," *The International Journal of Robotics Research*, vol. 4, no. 2, pp. 3–9, 1985.
- [9] J. Bhatia et al., "Reinforcement learning for freeform robot design," arXiv:2310.05670, 2023.
- [10] B. Tjanaka et al., "Co-optimization of morphology and behavior of modular robots via hierarchical deep reinforcement learning," in *Robotics: Science and Systems (RSS)*, 2023.
- [11] A. Spielberg et al., "Accelerated co-design of robots through morphological pretraining," arXiv:2502.10862, 2025.
- [12] M. Kalimuthu, A. A. Hayat, T. Pathmakumar, M. R. Elara, and K. L. Wood, "A deep reinforcement learning approach to optimal morphologies generation in reconfigurable tiling robots," *Mathematics*, vol. 11, no. 18, p. 3893, 2023.
- [13] Z. Ding, H. Tang, H. Wan, C. Zhang, and R. Sun, "A modular robotic arm configuration design method based on double dqn with prioritized experience replay," *Symmetry*, vol. 16, no. 6, p. 714, 2024. [Online]. Available: <https://www.mdpi.com/2073-8994/16/6/714>
- [14] K. S. Luck, H. B. Amor, R. Calandra, and J. Peters, "Data-efficient co-adaptation of morphology and behaviour with deep reinforcement learning," in *Conference on Robot Learning (CoRL)*, 2019.
- [15] PatentPC, "Top robotics vendors by market share and installations," Sep 2025. [Online]. Available: <https://patentpc.com/blog/top-robotics-vendors-by-market-share-installations>
- [16] Standard Bots, "Fanuc robot prices: Cost, models, and buying insights in 2025," 2025. [Online]. Available: <https://standardbots.com/blog/fanuc-robot-price>
- [17] S. Haddadin et al., "The franka emika robot: A reference platform for robotics research and education," *IEEE Robotics & Automation Magazine*, vol. 29, no. 2, pp. 46–64, 2022.
- [18] A. A. Oliva, F. Spindler, P. R. Giordano, and F. Chaumette, "Frankasim: A dynamic simulator for the franka emika robot with visual-servoing enabled capabilities," in *ICARCV 2022 - 17th International Conference on Control, Automation, Robotics and Vision*, Singapore, Singapore, Dec 2022, pp. 1–7.
- [19] C. A. Klein and B. E. Blaho, "Dexterity measures for the design and control of kinematically redundant manipulators," *The International Journal of Robotics Research*, vol. 6, no. 2, pp. 72–83, June 1987. [Online]. Available: <https://doi.org/10.1177/027836498700600206>
- [20] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.
- [21] F. America, "Lr mate 200id data sheet," 2024. [Online]. Available: <https://www.fanucamerica.com/docs/default-source/robotics-files/lr-mate/lr-mate-200id-data-sheet.pdf>
- [22] Franka Robotics GmbH, *Product Manual Franka Research 3*, release 1.0 ed., Mar 2024, document number: R02210. [Online]. Available: [https://franka.de/hubfs/Product%20Manual%20Franka%20Research%203\\_R02210-1.0-EN.pdf?hsLang=en](https://franka.de/hubfs/Product%20Manual%20Franka%20Research%203_R02210-1.0-EN.pdf?hsLang=en)